🔥 JVM Architecture Explained (Deep Dive)

The **Java Virtual Machine (JVM)** is the heart of Java's "Write Once, Run Anywhere" philosophy.
It provides **runtime execution**, **memory management**, **security**, and **performance optimization** for Java applications.

🧱 High-Level JVM Architecture

```
+-----------------------------------------------------+
|                    Java Application                 |
+-------------------------+---------------------------+
                          |
                          v
+-----------------------------------------------------+
|                    Class Loader                     |
+-----------------------------------------------------+
                          |
                          v
+-----------------------------------------------------+
|               Runtime Data Areas (Memory)           |
|  Method Area | Heap | Stack | PC | Native Stack     |
+-----------------------------------------------------+
                          |
                          v
+-----------------------------------------------------+
|             Execution Engine (JIT + GC)             |
+-----------------------------------------------------+
                          |
                          v
+-----------------------------------------------------+
|         Native Method Interface & Libraries         |
+-----------------------------------------------------+
```

1️⃣ ClassLoader Subsystem

🔹 What is ClassLoader?

The **ClassLoader** dynamically loads `.class` files into the JVM **at runtime**, not at compile time.

Java does **lazy loading** → classes are loaded **only when required**.

🔷 ClassLoader Types (Hierarchy)

```
      Bootstrap ClassLoader
                |
                v
Extension (Platform) ClassLoader
                |
                v
     Application ClassLoader
                |
                v
       Custom ClassLoaders
```

1. Bootstrap ClassLoader

- Loads **core Java classes**
  - Location:
    `JAVA_HOME/lib`
  - Examples:
    - `java.lang.*`
    - `java.util.*`
- Written in **native code (C/C++)**

2. Extension (Platform) ClassLoader

- Loads extension libraries
  - Location:
    - `JAVA_HOME/lib/ext`
  - Examples:
    - Java cryptography extensions

3. Application ClassLoader

- Loads **application-level classes**
  - From:
    - `classpath`
- Most common loader in enterprise apps

4. Custom ClassLoader

- User-defined
  - Used in:
    - Application servers
    - Plugin systems
    - Hot deployment

🔷 Class Loading Process (3 Phases)

`Loading → Linking → Initialization`

1 Loading

- Reads `.class` bytecode
- Creates `Class` object in Method Area

2 Linking

- **Verification** → bytecode safety
- **Preparation** → memory allocation for static variables
- **Resolution** → symbolic references → direct references

3 Initialization

- Executes:
  - `static blocks`
  - `static variables`

📌 Real Use Case: Application Servers

- **Tomcat / WebLogic**
- Each web application has its **own ClassLoader**
- Prevents class conflicts between applications

---

2 JVM Memory Areas (Runtime Data Areas)

JVM memory is divided into **thread-shared** and **thread-local** areas.

🧠 JVM Memory Diagram

```
+-------------------+  Thread Shared
        |   Method Area     |
        +-------------------+
        |       Heap        |
        +-------------------+

+-------------------+  Thread Local
        |     JVM Stack     |
        +-------------------+
        | Program Counter   |
```

```
+-------------------+
| Native Stack      |
+-------------------+
```

🟦 1. Method Area (Metaspace)

🔷 Stores:

- Class metadata
- Method metadata
- Static variables
- Runtime constant pool

🔷 Java 8 Change

Java ≤ 7	Java ≥ 8
PermGen	Metaspace
Fixed size	Grows dynamically

📌 **OutOfMemoryError: Metaspace** occurs if too many classes are loaded.

🧠 Use Case

- Large Spring Boot apps
- Excessive dynamic proxy creation (AOP, Hibernate)

🟦 2. Heap Memory

🔷 Stores:

- Objects
- Arrays
- Instance variables

🔷 Heap Structure

```
Heap
├── Young Generation
│   ├── Eden
│   ├── Survivor S0
│   └── Survivor S1
│
└── Old Generation
```

Young Generation

- Short-lived objects
- Frequent GC (Minor GC)

Old Generation

- Long-lived objects
- Infrequent but expensive GC (Major/Full GC)

📌 Real Use Case

- Web applications creating many request/response objects
- Microservices handling JSON parsing

🟦 3. JVM Stack

🔷 Stores:

- Method call frames
- Local variables
- Operand stack
- Return values

```
Thread Stack
┌──────────┐
│ Method C │
├──────────┤
│ Method B │
├──────────┤
│ Method A │
└──────────┘
```

📌 StackOverflowError

- Deep recursion

- Infinite method calls

🟦 4. Program Counter (PC) Register

🔷 Stores:

- Address of current instruction

📌 Each thread has its **own PC register**

🟦 5. Native Method Stack

🔷 Used for:

- JNI calls
- Native libraries (C/C++)

3️⃣ Garbage Collection (GC)

🔷 What is GC?

Garbage Collection automatically **frees memory** by removing **unreachable objects**.

Java uses **reachability analysis**, not reference counting.

🧠 GC Root Examples

- Local variables
- Active threads
- Static variables
- JNI references

🗑️ GC Flow Diagram

```
New Object
↓
Eden Space
↓ (Minor GC)
Survivor S0 ↔ Survivor S1
↓ (Promotion)
Old Generation
↓ (Major GC)
Memory Reclaimed
```

🧹 Types of Garbage Collection

Type	Area	Impact
Minor GC	Young Gen	Fast, frequent
Major GC	Old Gen	Slower
Full GC	Heap + Metaspace	Stop-the-world

🧪 Popular GC Algorithms

GC	Best For
Serial GC	Small apps
Parallel GC	Throughput
CMS	Low latency (deprecated)
G1 GC	Large heaps
ZGC	Ultra-low latency

📌 Default (Java 9+) → G1 GC

⚙️ Real JVM GC Flags

```
-Xms4g
-Xmx4g
-XX:+UseG1GC
-XX:MaxGCPauseMillis=200
-Xlog:gc*
```

🧠 Real Production Use Case

Banking / Payment System

- Large heap (16–32 GB)
- G1 or ZGC
- Low pause times
- Strict memory tuning

Microservices

- Small heap (512MB–2GB)
- Fast startup
- Frequent scaling

🧩 JVM Architecture Summary

Component	Purpose
ClassLoader	Loads classes dynamically
Method Area	Class metadata
Heap	Object storage
Stack	Method execution
GC	Automatic memory cleanup
Execution Engine	Interprets & compiles bytecode

🎯 Interview Tip (One-Liner)

JVM is a runtime engine that loads classes, manages memory, executes bytecode, and automatically handles garbage collection to ensure platform independence and performance.